

DBMS LAB – PRACTICAL P5

Aggregate and Grouping Queries – Using Aggregate Functions, GROUP BY, HAVING, ROLLUP and CUBE

Name	Sagar Kumar	PRN	SOE25BTAM29
Program	Bachelor of Technology	Section	A
Year	2	Semester	III
Course Code	U25AMILPC307	Course Title	DBMS Lab
Practical	P5	Database / DBMS	college_db / PostgreSQL 17

1. Aim

To write and execute aggregate and grouping queries on the College Management System database (**college_db**) in PostgreSQL 17 using the aggregate functions COUNT(), SUM(), AVG(), MIN() and MAX() together with the GROUP BY, HAVING, ROLLUP and CUBE clauses.

2. Objective

- To summarise many rows of a table into a single value using aggregate functions.
- To understand how aggregate functions treat NULL values (COUNT(*) vs COUNT(column)).
- To divide rows into groups with GROUP BY and compute one summary row per group.
- To filter groups with HAVING and to understand the difference between WHERE and HAVING.
- To produce sub-totals and grand totals in one query with GROUP BY ROLLUP.
- To produce totals for every combination of grouping columns with GROUP BY CUBE, and to identify super-aggregate rows with the GROUPING() function.

3. Theory

In a relational database, data is stored row by row, but reports usually need *summary* information – how many students are there, what is the average mark of a course, which department has the most students. SQL provides **aggregate functions** for this purpose. An aggregate function takes a set of rows as input and returns a single value as output.

3.1 GROUP BY

Without GROUP BY, an aggregate function treats the whole result as one group and returns exactly one row. The GROUP BY clause divides the rows into groups that have the same value(s) in the listed column(s); the aggregate is then computed separately for each group, giving one output row per group. Every column in the SELECT list must either appear in the GROUP BY clause or be used inside an aggregate function.

3.2 HAVING

WHERE filters individual rows *before* grouping, so it cannot contain aggregate functions. HAVING filters whole groups *after* the aggregates have been calculated, so conditions such as AVG(marks) > 75 must be written in HAVING. A query can use both: WHERE first removes unwanted rows, GROUP BY forms groups from the remaining rows and HAVING keeps only the groups that satisfy

the condition.

Logical order of evaluation of a SELECT statement:

```
FROM / JOIN -> WHERE -> GROUP BY -> aggregate functions -> HAVING
              -> SELECT -> ORDER BY -> LIMIT
```

3.3 ROLLUP

GROUP BY ROLLUP (a, b) produces normal groups and, in the same result, **hierarchical sub-totals** and a **grand total**. It works from right to left, removing one column at a time. ROLLUP of n columns produces $n + 1$ grouping sets. In the extra (super-aggregate) rows, the column that has been "rolled up" is shown as NULL. ROLLUP is used when the columns form a hierarchy, e.g. Department → Course, or Year → Month → Day.

3.4 CUBE

GROUP BY CUBE (a, b) produces totals for **every possible combination** of the listed columns. CUBE of n columns produces 2^n grouping sets. It is used for cross-tabulation (multi-dimensional analysis) when the columns are independent of each other, e.g. Department and Gender.

Clause	Equivalent GROUPING SETS	Rows generated for
GROUP BY a, b	((a, b))	each (a, b) pair only
GROUP BY ROLLUP (a, b)	((a, b), (a), (b))	each (a, b) pair + sub-total of each a + grand total
GROUP BY CUBE (a, b)	((a, b), (a), (b), (a, b))	each (a, b) pair + total of each a + total of each b + grand total

3.5 GROUPING() function

Because super-aggregate rows show NULL in the rolled-up column, it can be confused with a real NULL in the data. GROUPING(column) returns **1** when the column has been aggregated away in that row (sub-total / total row) and **0** when it is a normal group value. It is used with CASE to print meaningful labels such as 'Dept Subtotal' or 'GRAND TOTAL', and in ORDER BY to place total rows after the detail rows.

4. Aggregate Functions Explanation

Function	Purpose	NULL handling	Example (college_db)
COUNT(*)	Counts all rows in the group	Counts rows even if columns are NULL	COUNT(*) FROM student
COUNT(col)	Counts non-NULL values of a column	Ignores NULLs	COUNT(marks) FROM enrollment
COUNT(DISTINCT col)	Counts distinct non-NULL values	Ignores NULLs	COUNT(DISTINCT course_id)
SUM(col)	Adds all values (numeric columns)	Ignores NULLs	SUM(salary) FROM faculty
AVG(col)	Arithmetic mean = SUM / COUNT(col)	Ignores NULLs (not counted in the divisor)	AVG(marks) FROM enrollment

Function	Purpose	NULL handling	Example (college_db)
MIN(col)	Smallest value (numbers, dates, text)	Ignores NULLs	MIN(marks), MIN(date_of_birth)
MAX(col)	Largest value (numbers, dates, text)	Ignores NULLs	MAX(marks), MAX(salary)

On an empty input, COUNT returns 0 while SUM, AVG, MIN and MAX return NULL. ROUND(value, 2) is used with AVG() in this practical because AVG on a NUMERIC column returns many decimal places.

5. SQL Queries

5.1 Database and Environment

The same College Management System database **college_db** used in P1–P4 is used here. It contains five related tables:

Table	Columns (PK underlined)	Rows
department	<u>department_id</u> , department_name, hod_name, building	5
faculty	<u>faculty_id</u> , faculty_name, email, designation, salary, department_id (FK)	8
student	<u>student_id</u> , first_name, last_name, email, gender, date_of_birth, year_of_study, department_id (FK)	15
course	<u>course_id</u> , course_code, course_name, credits, department_id (FK), faculty_id (FK)	8
enrollment	<u>enrollment_id</u> , student_id (FK), course_id (FK), enrollment_date, marks	29

Two details of the data are useful for this practical: the **Civil** department has no students yet, and one enrollment (Arjun Malhotra in Machine Learning) has **marks = NULL** because the result is awaited. These show how aggregates handle empty groups and NULL values.

Environment: PostgreSQL 17.11 server, queries typed in the **psql** command-line client connected to college_db. All screenshots below were captured from this live psql session. The complete scripts are 01_college_db_setup.sql (tables and sample data) and 02_p5_aggregate_queries.sql (the queries below).

5.2 COUNT(), SUM() and AVG() (Queries 1 – 4)

```
-- Q1. Total number of students
SELECT COUNT(*) AS total_students
FROM student;
```

```
-- Q2. COUNT(*) vs COUNT(column): COUNT(marks) ignores NULL marks
SELECT COUNT(*)      AS total_enrollments,
       COUNT(marks) AS graded_enrollments
FROM enrollment;
```

```
-- Q3. Total monthly salary expense of all faculty
SELECT SUM(salary) AS total_salary_expense
FROM faculty;
```

```
-- Q4. Average marks (rounded to 2 decimal places)
SELECT ROUND(AVG(marks), 2) AS average_marks
FROM enrollment;
```

5.3 MIN() and MAX() (Queries 5 – 8)

```
-- Q5. Highest marks
SELECT MAX(marks) AS highest_marks
FROM enrollment;
```

```
-- Q6. Lowest marks
SELECT MIN(marks) AS lowest_marks
FROM enrollment;
```

```
-- Q7. Salary range of faculty
SELECT MIN(salary) AS min_salary,
       MAX(salary) AS max_salary,
       MAX(salary) - MIN(salary) AS salary_gap
FROM faculty;
```

```
-- Q8. Youngest and oldest student (MIN/MAX on a DATE column)
SELECT MIN(date_of_birth) AS oldest_student_dob,
       MAX(date_of_birth) AS youngest_student_dob
FROM student;
```

5.4 GROUP BY (Queries 9 – 10)

```
-- Q9. Department-wise student count
SELECT d.department_name,
       COUNT(s.student_id) AS total_students
FROM department d
LEFT JOIN student s
      ON d.department_id = s.department_id
GROUP BY d.department_name
ORDER BY total_students DESC, d.department_name;
```

```
-- Q10. Average, highest and lowest marks per course
SELECT c.course_name,
       COUNT(e.marks)           AS students_graded,
       ROUND(AVG(e.marks), 2) AS average_marks,
       MAX(e.marks)            AS highest,
       MIN(e.marks)            AS lowest
FROM course c
JOIN enrollment e
      ON c.course_id = e.course_id
GROUP BY c.course_name
ORDER BY average_marks DESC;
```

5.5 HAVING (Queries 11 – 12)

```
-- Q11. Courses whose average marks are greater than 75
SELECT c.course_name,
       ROUND(AVG(e.marks), 2) AS average_marks
FROM course c
JOIN enrollment e
      ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING AVG(e.marks) > 75
ORDER BY average_marks DESC;
```

```
-- Q12. Departments having at least 3 second-year students
--      (WHERE filters rows first, HAVING then filters the groups)
SELECT d.department_name,
       COUNT(*) AS second_year_students
FROM department d
JOIN student s
      ON d.department_id = s.department_id
WHERE s.year_of_study = 2
GROUP BY d.department_name
HAVING COUNT(*) >= 3
ORDER BY second_year_students DESC;
```

5.6 ROLLUP (Queries 13 – 14)

```
-- Display NULLs visibly so the super-aggregate rows are easy to spot
\pset null '(null)'
```

```
-- Q13. Student count by department and year of study, with ROLLUP.
--      (null) in year_of_study = subtotal for that department
--      (null) in both columns  = grand total
SELECT d.department_name,
       s.year_of_study,
       COUNT(*) AS total_students
FROM student s
JOIN department d ON s.department_id = d.department_id
GROUP BY ROLLUP (d.department_name, s.year_of_study)
ORDER BY d.department_name, s.year_of_study;
```

```
-- Q14. Enrollments and average marks per course, with a subtotal for
--      every department and one grand total row (labelled using GROUPING()).
SELECT CASE WHEN GROUPING(d.department_name) = 1 THEN '** GRAND TOTAL **'
          ELSE d.department_name END           AS department,
       CASE WHEN GROUPING(d.department_name) = 1 THEN '-- All Courses --'
          WHEN GROUPING(c.course_name) = 1      THEN '-- Dept Subtotal --'
          ELSE c.course_name END                AS course,
       COUNT(e.enrollment_id)                  AS enrollments,
       ROUND(AVG(e.marks), 2)                   AS avg_marks
FROM enrollment e
JOIN course c   ON e.course_id = c.course_id
JOIN department d ON c.department_id = d.department_id
GROUP BY ROLLUP (d.department_name, c.course_name)
ORDER BY GROUPING(d.department_name), d.department_name,
         GROUPING(c.course_name), c.course_name;
```

5.7 CUBE (Query 15)

```
-- Q15. Student count for every combination of department and gender,
--      including department totals, gender totals and a grand total.
SELECT COALESCE(d.department_name, 'ALL DEPARTMENTS') AS department,
       CASE WHEN GROUPING(s.gender) = 1 THEN 'ALL'
          ELSE s.gender END                          AS gender,
       COUNT(*)                                       AS total_students
FROM student s
JOIN department d ON s.department_id = d.department_id
GROUP BY CUBE (d.department_name, s.gender)
ORDER BY GROUPING(d.department_name), d.department_name,
         GROUPING(s.gender), s.gender;
```

6. Actual Output

The following screenshots were captured from the psql client while executing the above queries on PostgreSQL 17.11. Each screenshot shows the query exactly as typed at the college_db=# prompt and the rows returned by the server.

```
psql (17.11 (Debian 17.11-1.pgdg13+2))
Type "help" for help.

college_db=# SELECT version();
               version
-----
PostgreSQL 17.11 (Debian 17.11-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)

college_db=# \dt
      List of relations
Schema | Name          | Type  | Owner
-----+-----+-----+-----
public | course        | table | postgres
public | department    | table | postgres
public | enrollment    | table | postgres
public | faculty       | table | postgres
public | student       | table | postgres
(5 rows)

college_db=#
```

Screenshot 0: psql connected to college_db – server version (PostgreSQL 17.11) and the five tables (dt)

```
college_db=# -- Q1. Total number of students
college_db=# SELECT COUNT(*) AS total_students
college_db=# FROM student;
 total_students
-----
              15
(1 row)

college_db=# -- Q2. COUNT(*) vs COUNT(column): COUNT(marks) ignores NULL marks
college_db=# SELECT COUNT(*)      AS total_enrollments,
college_db=#      COUNT(marks) AS graded_enrollments
college_db=# FROM enrollment;
 total_enrollments | graded_enrollments
-----+-----
                29 |                28
(1 row)

college_db=# -- Q3. Total monthly salary expense of all faculty
college_db=# SELECT SUM(salary) AS total_salary_expense
college_db=# FROM faculty;
 total_salary_expense
-----
             835000.00
(1 row)

college_db=# -- Q4. Average marks (rounded to 2 decimal places)
college_db=# SELECT ROUND(AVG(marks), 2) AS average_marks
college_db=# FROM enrollment;
 average_marks
-----
           75.54
(1 row)

college_db=#
```

Screenshot 1: COUNT, SUM and AVG results (psql, PostgreSQL 17.11)

```

college_db=# -- Q5. Highest marks
college_db=# SELECT MAX(marks) AS highest_marks
college_db=# FROM enrollment;
highest_marks
-----
          94.00
(1 row)

college_db=# -- Q6. Lowest marks
college_db=# SELECT MIN(marks) AS lowest_marks
college_db=# FROM enrollment;
lowest_marks
-----
          55.00
(1 row)

college_db=# -- Q7. Salary range of faculty
college_db=# SELECT MIN(salary) AS min_salary,
college_db=#          MAX(salary) AS max_salary,
college_db=#          MAX(salary) - MIN(salary) AS salary_gap
college_db=# FROM faculty;
min_salary | max_salary | salary_gap
-----+-----+-----
 68000.00 | 130000.00 |   62000.00
(1 row)

college_db=# -- Q8. Youngest and oldest student (MIN/MAX on a DATE column)
college_db=# SELECT MIN(date_of_birth) AS oldest_student_dob,
college_db=#          MAX(date_of_birth) AS youngest_student_dob
college_db=# FROM student;
oldest_student_dob | youngest_student_dob
-----+-----
2005-05-29         | 2006-12-01
(1 row)

college_db=# █

```

Screenshot 2: MIN and MAX results (psql, PostgreSQL 17.11)

```

college_db=# -- Q9. Department-wise student count
college_db=# SELECT d.department_name,
college_db=#          COUNT(s.student_id) AS total_students
college_db=# FROM department d
college_db=# LEFT JOIN student s
college_db=#          ON d.department_id = s.department_id
college_db=# GROUP BY d.department_name
college_db=# ORDER BY total_students DESC, d.department_name;
department_name | total_students
-----+-----
AI and Machine Learning |          5
Computer Science      |          4
Electronics           |          3
Mechanical             |          3
Civil                  |          0
(5 rows)

college_db=# █

```

Screenshot 3(a): Department-wise student count using GROUP BY

```
college_db=# -- Q10. Average, highest and lowest marks per course
college_db=# SELECT c.course_name,
college_db=#         COUNT(e.marks)           AS students_graded,
college_db=#         ROUND(AVG(e.marks), 2) AS average_marks,
college_db=#         MAX(e.marks)            AS highest,
college_db=#         MIN(e.marks)            AS lowest
college_db=# FROM course c
college_db=# JOIN enrollment e
college_db=#   ON c.course_id = e.course_id
college_db=# GROUP BY c.course_name
college_db=# ORDER BY average_marks DESC;
  course_name | students_graded | average_marks | highest | lowest
-----+-----+-----+-----+-----
Machine Learning | 4 | 83.38 | 91.00 | 73.00
Database Management | 6 | 80.75 | 94.00 | 68.00
Data Structures | 4 | 80.75 | 92.00 | 65.00
Digital Electronics | 3 | 72.50 | 81.50 | 62.00
Thermodynamics | 3 | 71.17 | 84.00 | 59.00
Operating Systems | 3 | 70.67 | 81.00 | 58.50
Signals and Systems | 3 | 67.17 | 77.00 | 55.00
Structural Analysis | 2 | 64.75 | 66.00 | 63.50
(8 rows)

college_db=#
```

Screenshot 3(b): Course-wise average, highest and lowest marks using GROUP BY

```
college_db=# -- Q11. Courses whose average marks are greater than 75
college_db=# SELECT c.course_name,
college_db=#         ROUND(AVG(e.marks), 2) AS average_marks
college_db=# FROM course c
college_db=# JOIN enrollment e
college_db=#   ON c.course_id = e.course_id
college_db=# GROUP BY c.course_name
college_db=# HAVING AVG(e.marks) > 75
college_db=# ORDER BY average_marks DESC;
  course_name | average_marks
-----+-----
Machine Learning | 83.38
Database Management | 80.75
Data Structures | 80.75
(3 rows)

college_db=# -- Q12. Departments having at least 3 second-year students
college_db=# -- (WHERE filters rows first, HAVING then filters the groups)
college_db=# SELECT d.department_name,
college_db=#         COUNT(*) AS second_year_students
college_db=# FROM department d
college_db=# JOIN student s
college_db=#   ON d.department_id = s.department_id
college_db=# WHERE s.year_of_study = 2
college_db=# GROUP BY d.department_name
college_db=# HAVING COUNT(*) >= 3
college_db=# ORDER BY second_year_students DESC;
 department_name | second_year_students
-----+-----
AI and Machine Learning | 4
Computer Science | 3
(2 rows)

college_db=#
```

Screenshot 4: Filtering groups using HAVING

```

college_db=# -- Display NULLs visibly so the super-aggregate rows are easy to spot
college_db=# \pset null '(null)'
Null display is "(null)".
college_db=# -- Q13. Student count by department and year of study, with ROLLUP.
college_db=# --      (null) in year_of_study = subtotal for that department
college_db=# --      (null) in both columns = grand total
college_db=# SELECT d.department_name,
college_db=#      s.year_of_study,
college_db=#      COUNT(*) AS total_students
college_db=# FROM student s
college_db=# JOIN department d ON s.department_id = d.department_id
college_db=# GROUP BY ROLLUP (d.department_name, s.year_of_study)
college_db=# ORDER BY d.department_name, s.year_of_study;
  department_name | year_of_study | total_students
-----+-----+-----
AI and Machine Learning | 2 | 4
AI and Machine Learning | 3 | 1
AI and Machine Learning | (null) | 5
Computer Science | 2 | 3
Computer Science | 3 | 1
Computer Science | (null) | 4
Electronics | 2 | 2
Electronics | 3 | 1
Electronics | (null) | 3
Mechanical | 2 | 2
Mechanical | 3 | 1
Mechanical | (null) | 3
(null) | (null) | 15
(13 rows)

college_db=# █

```

Screenshot 5(a): GROUP BY ROLLUP with super-aggregate rows shown as (null)

```

college_db=# -- Q14. Enrollments and average marks per course, with a subtotal for
college_db=# --      every department and one grand total row (labelled using GROUPING()).
college_db=# SELECT CASE WHEN GROUPING(d.department_name) = 1 THEN '** GRAND TOTAL **'
college_db=#      ELSE d.department_name END AS department,
college_db=#      CASE WHEN GROUPING(d.department_name) = 1 THEN '-- All Courses --'
college_db=#      WHEN GROUPING(c.course_name) = 1 THEN '-- Dept Subtotal --'
college_db=#      ELSE c.course_name END AS course,
college_db=#      COUNT(e.enrollment_id) AS enrollments,
college_db=#      ROUND(AVG(e.marks), 2) AS avg_marks
college_db=# FROM enrollment e
college_db=# JOIN course c ON e.course_id = c.course_id
college_db=# JOIN department d ON c.department_id = d.department_id
college_db=# GROUP BY ROLLUP (d.department_name, c.course_name)
college_db=# ORDER BY GROUPING(d.department_name), d.department_name,
college_db=#      GROUPING(c.course_name), c.course_name;
  department | course | enrollments | avg_marks
-----+-----+-----+-----
AI and Machine Learning | Database Management | 6 | 80.75
AI and Machine Learning | Machine Learning | 5 | 83.38
AI and Machine Learning | -- Dept Subtotal -- | 11 | 81.80
Civil | Structural Analysis | 2 | 64.75
Civil | -- Dept Subtotal -- | 2 | 64.75
Computer Science | Data Structures | 4 | 80.75
Computer Science | Operating Systems | 3 | 70.67
Computer Science | -- Dept Subtotal -- | 7 | 76.43
Electronics | Digital Electronics | 3 | 72.50
Electronics | Signals and Systems | 3 | 67.17
Electronics | -- Dept Subtotal -- | 6 | 69.83
Mechanical | Thermodynamics | 3 | 71.17
Mechanical | -- Dept Subtotal -- | 3 | 71.17
** GRAND TOTAL ** | -- All Courses -- | 29 | 75.54
(14 rows)

college_db=# █

```

Screenshot 5(b): GROUP BY ROLLUP with sub-total and grand-total rows labelled using GROUPING()

```

college_db=# -- Q15. Student count for every combination of department and gender,
college_db=# -- including department totals, gender totals and a grand total.
college_db=# SELECT COALESCE(d.department_name, 'ALL DEPARTMENTS') AS department,
college_db=# CASE WHEN GROUPING(s.gender) = 1 THEN 'ALL'
college_db=# ELSE s.gender END AS gender,
college_db=# COUNT(*) AS total_students
college_db=# FROM student s
college_db=# JOIN department d ON s.department_id = d.department_id
college_db=# GROUP BY CUBE (d.department_name, s.gender)
college_db=# ORDER BY GROUPING(d.department_name), d.department_name,
college_db=# GROUPING(s.gender), s.gender;
 department | gender | total_students
-----+-----+-----
 AI and Machine Learning | F      | 2
 AI and Machine Learning | M      | 3
 AI and Machine Learning | ALL    | 5
 Computer Science      | F      | 2
 Computer Science      | M      | 2
 Computer Science      | ALL    | 4
 Electronics           | F      | 2
 Electronics           | M      | 1
 Electronics           | ALL    | 3
 Mechanical            | F      | 1
 Mechanical            | M      | 2
 Mechanical            | ALL    | 3
 ALL DEPARTMENTS      | F      | 7
 ALL DEPARTMENTS      | M      | 8
 ALL DEPARTMENTS      | ALL    | 15
(15 rows)

college_db=# █

```

Screenshot 6: GROUP BY CUBE – department × gender totals

7. Query Explanation

7.1 COUNT(), SUM() and AVG() (Queries 1 – 4) – Screenshot 1

- **Q1** – COUNT(*) counts every row of the student table: **15** students.
- **Q2** – COUNT(*) returns **29** enrollments, but COUNT(marks) returns **28** because COUNT(column) skips the one enrollment whose marks are NULL.
- **Q3** – SUM(salary) adds the salaries of all 8 faculty members: **835000.00**.
- **Q4** – AVG(marks) = 75.54. The NULL mark is ignored, so the average is taken over the 28 graded enrollments (SUM / 28, not SUM / 29). ROUND(..., 2) keeps two decimal places.

7.2 MIN() and MAX() (Queries 5 – 8) – Screenshot 2

- **Q5 / Q6** – The highest mark in any course is **94.00** and the lowest is **55.00**.
- **Q7** – Several aggregates can be used in one SELECT and combined in expressions: salaries range from 68000.00 to 130000.00, a gap of 62000.00.
- **Q8** – MIN and MAX also work on DATE columns. The smallest date of birth (2005-05-29) belongs to the oldest student and the largest (2006-12-01) to the youngest.

7.3 GROUP BY (Queries 9 – 10) – Screenshot 3(a), Screenshot 3(b)

- **Q9** – Students are grouped by department and counted. A LEFT JOIN is used so that the Civil department, which has no students, still appears. COUNT(s.student_id) is used instead of COUNT(*) so that Civil shows **0**; COUNT(*) would wrongly show 1 because the LEFT JOIN creates one row with NULL student columns.
- **Q10** – Enrollment rows are grouped by course and five aggregates are computed per group. Machine Learning has the best average (83.38) and shows students_graded = 4 because the NULL mark is not counted. Structural Analysis has the lowest average (64.75).

7.4 HAVING (Queries 11 – 12) – Screenshot 4

- **Q11** – After grouping by course, HAVING AVG(e.marks) > 75 keeps only the courses whose average is above 75. Out of 8 courses, 3 qualify: Machine Learning (83.38), Database Management (80.75) and Data Structures (80.75).
- **Q12** – Shows WHERE and HAVING together. WHERE year_of_study = 2 first removes third-year students, GROUP BY counts the remaining students per department and HAVING COUNT(*) >= 3 keeps only departments with at least three second-year students: AI and Machine Learning (4) and Computer Science (3).
- An aggregate cannot be written in WHERE – PostgreSQL rejects it with *ERROR: aggregate functions are not allowed in WHERE*; such conditions belong in HAVING.

7.5 ROLLUP (Queries 13 – 14) – Screenshot 5(a), Screenshot 5(b)

- \pset null '(null)' is a psql command that only changes how NULL is displayed, so that the rows added by ROLLUP are easy to see.
- **Q13** – ROLLUP (department_name, year_of_study) generates the grouping sets (department, year), (department) and (). So for each department we get one row per year, then a department sub-total row where year_of_study is (null) (e.g. AI and Machine Learning = 5), and finally one grand-total row where both columns are (null) with **15** students. 8 detail rows + 4 sub-totals + 1 grand total = 13 rows.

- **Q14** – ROLLUP over the hierarchy Department → Course. GROUPING() returns 1 for a rolled-up column, and CASE uses it to print '-- Dept Subtotal --' and '*** GRAND TOTAL ***' instead of NULL. The ORDER BY on GROUPING() places each sub-total after its courses and the grand total last.
- Sub-total and total averages are calculated from the underlying rows, not by averaging the course averages: AI and Machine Learning = 81.80 over its 10 graded enrollments, and the grand total 75.54 equals the overall average from Q4. The enrollments column counts all 29 enrollments (COUNT(e.enrollment_id)), while AVG() ignores the one NULL mark.

7.6 CUBE (Query 15) – Screenshot 6

- **Q15** – CUBE (department_name, gender) generates $2^2 = 4$ grouping sets: (department, gender), (department), (gender) and ().
- Rows with a real gender value give the count for that department and gender; rows with gender = 'ALL' are department totals; rows with 'ALL DEPARTMENTS' and F / M are gender totals across the college (7 female, 8 male); the last row is the grand total (15).
- The (gender) totals – 'ALL DEPARTMENTS / F' and 'ALL DEPARTMENTS / M' – are exactly the rows that ROLLUP would *not* produce. This is the difference between CUBE and ROLLUP.
- COALESCE replaces the NULL department in the super-aggregate rows with 'ALL DEPARTMENTS' (safe here because department_name is NOT NULL), and GROUPING() is used for the gender label. Civil does not appear because the INNER JOIN keeps only departments that have students.

8. Result

Aggregate and grouping queries were successfully written and executed on the **college_db** College Management System database in PostgreSQL 17.11. COUNT(), SUM(), AVG(), MIN() and MAX() were used to summarise student, faculty and enrollment data; GROUP BY was used to produce department-wise and course-wise summaries; HAVING was used to filter groups based on aggregate conditions; and GROUP BY ROLLUP and GROUP BY CUBE were used to generate sub-totals, cross-totals and grand totals in a single query, with GROUPING() used to label the super-aggregate rows. All outputs were verified against the actual results returned by PostgreSQL.

9. Viva Questions and Answers

Q1. What is an aggregate function?

Ans: A function that takes a set of rows as input and returns a single summary value, e.g. COUNT, SUM, AVG, MIN and MAX.

Q2. What is the difference between COUNT(*), COUNT(column) and COUNT(DISTINCT column)?

Ans: COUNT(*) counts all rows including those with NULLs; COUNT(column) counts only the non-NULL values of that column; COUNT(DISTINCT column) counts unique non-NULL values. In college_db, COUNT(*) FROM enrollment = 29 but COUNT(marks) = 28.

Q3. How do aggregate functions handle NULL values?

Ans: All aggregate functions except COUNT(*) ignore NULLs. AVG(marks) therefore divides by the number of non-NULL marks (28), not by the number of rows (29).

Q4. What is the difference between WHERE and HAVING?

Ans: WHERE filters individual rows before grouping and cannot use aggregate functions. HAVING filters groups after GROUP BY and aggregation, so it can use aggregates such as AVG(marks) > 75.

Q5. Can an aggregate function be used in the WHERE clause?

Ans: No. PostgreSQL gives the error "aggregate functions are not allowed in WHERE". The condition must be written in HAVING (or in a subquery).

Q6. What rule applies to the SELECT list when GROUP BY is used?

Ans: Every selected column must either appear in GROUP BY or be inside an aggregate function. (PostgreSQL also allows columns that are functionally dependent on a grouped primary key.)

Q7. Can HAVING be used without GROUP BY?

Ans: Yes. The whole table is then treated as one group, e.g. SELECT COUNT(*) FROM student HAVING COUNT(*) > 10;

Q8. What is the logical order of execution of a SELECT query with grouping?

Ans: FROM/JOIN → WHERE → GROUP BY → aggregates → HAVING → SELECT → ORDER BY → LIMIT.

Q9. What does GROUP BY ROLLUP do?

Ans: It adds hierarchical sub-totals and a grand total to the normal grouped result. ROLLUP (a, b) = GROUPING SETS ((a, b), (a), ()); for n columns it creates n + 1 grouping sets.

Q10. What does GROUP BY CUBE do?

Ans: It produces aggregates for every combination of the grouping columns. CUBE (a, b) = GROUPING SETS ((a, b), (a), (b), ()); for n columns it creates 2ⁿ grouping sets.

Q11. What is the difference between ROLLUP and CUBE?

Ans: ROLLUP is hierarchical and depends on column order (sub-totals only from right to left); CUBE is non-hierarchical and gives totals for all combinations. For two columns, CUBE gives the extra (b) totals – e.g. gender totals across all departments – which ROLLUP does not.

Q12. What is the GROUPING() function?

Ans: GROUPING(column) returns 1 if the column was aggregated away in that row (a sub-total or total row) and 0 otherwise. It distinguishes the NULL produced by ROLLUP/CUBE from a real NULL in the data and is used to label total rows.

Q13. What is GROUPING SETS?

Ans: A clause that lists exactly which groupings to compute in one query, e.g. GROUP BY GROUPING SETS ((department_name), (gender), ()). ROLLUP and CUBE are shorthand forms of GROUPING SETS.

Q14. Why was LEFT JOIN with COUNT(s.student_id) used for the department-wise count?

Ans: The LEFT JOIN keeps departments that have no students (Civil). COUNT(s.student_id) returns 0 for Civil because its student_id is NULL, whereas COUNT(*) would return 1.

Q15. Why is ROUND() used with AVG()?

Ans: AVG() on a NUMERIC column returns a value with many decimal places (e.g. 75.5357142857142857); ROUND(AVG(marks), 2) makes it readable as 75.54.